Iterative Prompt Revision - A Matter of Difficulty

May 3, 2026

1 Iterative Prompt Revision - A Matter of Difficulty

Author: Andrew Hewitt

1.0.1 Introduction:

MathTrain is a mathematics practice website that generates undergraduate-level practice problems based on user-defined categories. As part of its functionality, it provides walkthroughs that are generated alongside each practice problem. Seeking higher-quality walkthroughs, the previous paper established a pipeline for iterative improvement of the walkthrough prompt. It identified a possible pain point in this process: iterative prompt revision may overcorrect for minor problems, ultimately hurting the quality of front-facing responses. This paper expands upon the process found in the last, implementing more controls on variability and a higher level of automation. The dataset generated for this paper is significantly larger than the previous one, offering enhanced capabilities for in-depth analysis. Thus, this paper aims to identify patterns in overcorrection and explores results of the newer, more hands-off data generation process. Additionally, it aims to discover a new production prompt for the MathTrain website.

1.0.2 Research objectives:

Particularly, this study aims to answer the following questions: - Is there a pattern in the overcorrection (oscillation, singular peak, etc.)? - If so, what are the characteristics of the pedagogical score fluctuations (Period, max & min score, etc.)? - What causes errors in problem generation?

1.0.3 Methodology:

Generating Data: Data creation and prompt revision are now unified into a single, easy process. The generation of responses, grades, and prompts are handled by a Command Line Interface (CLI), from which the user can select how many batches they would like to create, as well as how many responses they would like per batch.

Assessing Responses: To ensure the quality of the responses, a rubric was developed in the last project to assess the teaching quality of each walkthrough. This rubric can be found in the file 'rubric.txt'. The rubric, along with a batch of responses to the prompt are fed into a grading Large Language Model (LLM). From there, the grader scores the responses based on the rubric, and provides prompt revision feedback for each individual response. Each response is graded one-by-one, batchwise. Grades and feedback are compiled into a dedicated csv file for analysis.

Paired Testing: To ensure a statistically reliable basis for analysis, the CLI-generation scheme uses paired testing for the problem parameters. Essentially, the generating program selects random topics and difficulties, which are stored so that each batch has uniform problem parameters. This garuntees that one batch doesn't feed a more difficult problem to the walkthrough-generating LLM; it creates walkthroughs for the same problem types and difficulties every time.

Automated Revisions: With feedback in place, this project implements a new framework for automated prompt revision. After each batch is generated, grading feedback is automatically fed into a revision LLM, which edits the prompt of the current batch, which is then placed into a csv file containing the prompts used for each batch. When the next batch is generated, it pulls the most recent prompt from the csv file (the one generated from the feedback of the previous prompt).

Limitations: The current pipeline does not account for JSON formatting errors in responses from LLMs. As such, data loss can occur, especially in cases where the editing LLM tweaks the formatting rules. In this paper, we observe where these shortcomings are most likely to occur.

Project Specifics: This project uses a database comprised of 20 batches with 40 problems each. Because of formatting errors, the total entries falls under the expected 800 at 769. Factors of data loss and formatting errors are identified in the first section of the exploratory data analysis.

Tips for Reproduction: Since data generation is done through a CLI on terminal, it is suggested that the computer running the generation has automatic sleep mode turned off during larger generation sessions. This prevents errors and shortens the total time of data generation.

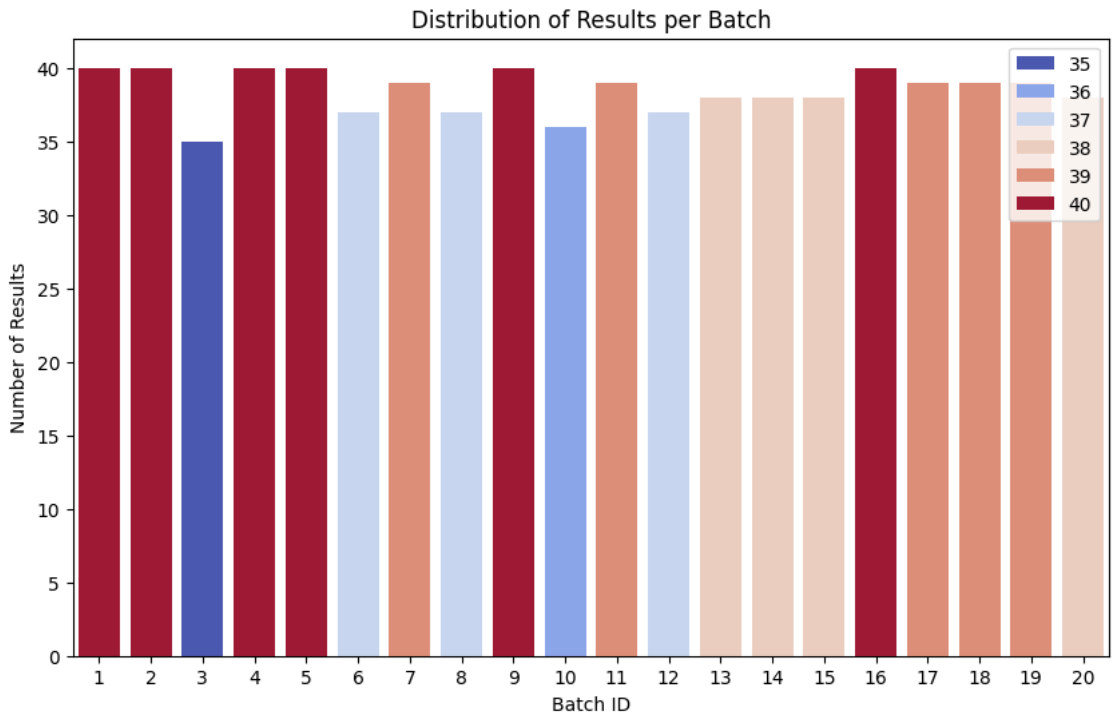
1.0.4 Imports, Dataframe acquisition, and Cleaning

In the data generation process, the grading LLM occasionally entered hallucinated values when calculating the percentage grade. Percentages were manually calculated to ensure uniform correctness.

1.0.5 Exploratory Data Analysis:

Data Loss Analysis: Some responses in this generation scheme didn't follow proper JSON formatting, which caused an error that excluded them from the final csv file.

```
Text(0, 0.5, 'Number of Results')
```



According to the generation parameters set at the beginning of data collection, each batch should have 40 problems. However, because of JSON formatting errors, some batches have less than expected. These gaps in data can provide key insights about which prompts might have poor formatting instructions. For instance batch 3 only has 35 responses.

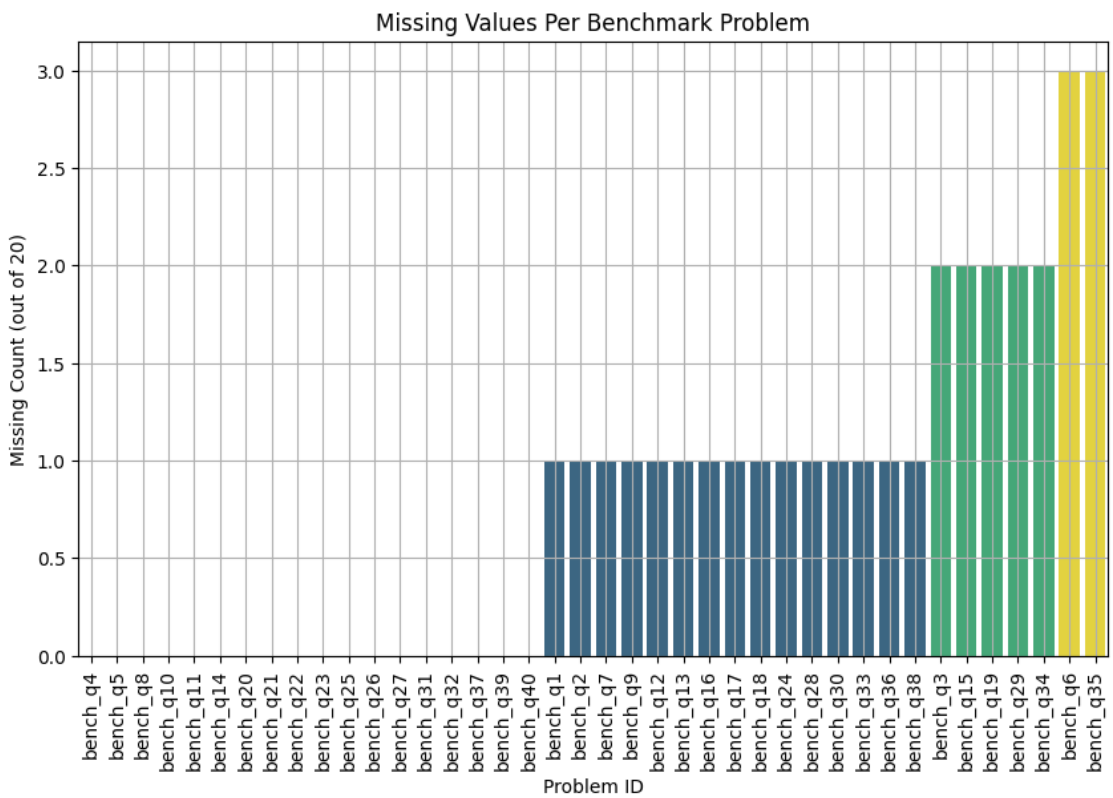
```
'You are a mathematics professor. Respond with valid JSON only.\n  FORMAT: {
"topic": "topic-key", "difficulty": "easy|medium|hard", "question": "$$LaTeX$$",
"solution": "Markdown" }\n  RULES FOR QUESTION FIELD: Questions must be wrapped
in $$…$$. If question is mixed with standard text, standard text should be
above the LaTeX, which should be wrapped in $$…$$. Do not end with
punctuation. Use colons to separate question text from the LaTeX if both are
present.\n  RULES FOR SOLUTION FIELD: Solution uses bold and numbered steps.
Display in solution should be wrapped in $$…$$. Inline latex should be wrapped
in $…$. A mix of text and LaTeX (display and inline) should be used in the
solution to naturally explain the reasoning.\n  PEDAGOGICAL REQUIREMENTS:\n  -
Justify method selection explicitly: explain why this technique is optimal and
compare to alternatives when relevant, but do so concisely without exploring
messy dead ends. If alternatives are mentioned, keep them brief and focused.\n
- Connect algebraic manipulations to strategic problem-solving: highlight why
specific factorizations, substitutions, or simplifications matter.\n  - Provide
geometric or physical interpretations: connect abstract calculations to visual
meaning, spatial relationships, or real-world context.\n  - Explain foundational
formulas: derive or intuitively explain key formulas (e.g., dS, cross products,
surface area elements) rather than assuming knowledge.\n  - Address domain
```

constraints: discuss domain restrictions, absolute values, and potential extraneous solutions. For trig substitution, explicitly discuss the range of and how absolute values are handled (e.g., using cos 0 for appropriate intervals).\n - Include verification or validation: check solutions in original equations, verify geometric intuition, or compare to known formulas when applicable. For trig substitution, suggest verifying by differentiation.\n - Connect to fundamental concepts: relate results to underlying theory (e.g., limits, derivatives, geometric series) to deepen understanding. For example, explain why the second derivative test confirms global minimum by noting function behavior (e.g., quadratic or convexity), explain why a row of zeros with non-zero constant implies inconsistency (0 = constant is impossible), and justify termwise integration of power series by uniform convergence on intervals of convergence.\n - Balance rigor with conciseness: avoid unnecessary verbosity while maintaining mathematical completeness. Keep conceptual explanations focused and avoid overextending domain discussions. Present the cleanest solution directly without exploring dead ends; if alternatives are discussed, do so in one sentence.\n - Provide conceptual bridging: explain why bounds are chosen (e.g., for cylindrical coordinates, justify z-bounds: below the cone z=r and above z=0). For trig substitution, explicitly state the substitution used and why it simplifies the integral. For line integrals, explain why ds = |r\'(t)| dt as the arc length differential and connect to the idea of mass of a wire. For definite integrals, explain the Fundamental Theorem of Calculus connection between antiderivative and area. For gradient, mention its role in directional derivatives. For dot product, explain why negative implies obtuse angle (cos < 0 in (/2,)). For projection, explain geometric meaning of scalar projection (component) vs vector projection. For hyperbolas, show how asymptotes follow from setting equation to zero. For symmetric orthogonal matrices, justify A^2 = I using A^T A = I and A^T = A, with reference to spectral theorem if appropriate. For inverse trig, explain quadrant selection (e.g., sine negative in fourth quadrant). For chain rule, justify differentiation of composite functions (outer derivative times inner derivative). For series, state if terms are positive, show limit computation detail, and explain why limit comparison test yields same convergence behavior. For logarithms, connect product rule to exponent properties. For functions, explain why radicand must be non-negative and denominator non-zero. For eigenvalues, verify left eigenvectors explicitly using A^T w = w. For basis-and-dimension, justify why row space equals span of vectors. For chain rule multivariable, explain why partial derivatives are necessary. For optimization, explain why only critical point yields global minimum due to function behavior (e.g., quadratic form). For rational expressions, explain why contradiction implies no solution and discuss extraneous solutions if cancellation occurred. For transformation (Jacobian), specify that transformation T is given by Φ and clarify domain mapping. For scalar triple product, explicitly connect to determinant and volume.\n - CRITICAL TOPIC ROUTE CONSTRAINT: The solution must strictly adhere to the topic specified in the input. For example, if the topic is \'vector-operations\', all reasoning must be grounded in vector operations (e.g., dot product, cross product, scalar triple product, vector magnitude), and

```
explicit connections to that topic must be made throughout the solution. Avoid
introducing unrelated topics (e.g., Jacobian) unless explicitly tying them back
to the specified topic (e.g., relating Jacobian to cross product as an
area/volume scaling factor).\n  - When using scalar triple product, explicitly
connect it to the determinant and volume: explain that the scalar triple product
$|(\\mathbf{a} \\times \\mathbf{b}) \\cdot \\mathbf{c}|$ equals the absolute
value of the determinant formed by the vectors, which geometrically gives the
volume of the parallelepiped spanned by them.\n  - Do NOT reference any other
topic or session ID in the solution. The solution must be self-contained and
consistent with the given topic only.\n  Do NOT include any text outside the
JSON.'
```

Prompt 3 is a highly complex prompt with a lot of specific instructions. The JSON formatting errors could be indicative of an "instruction gap" where an LLM struggles to deal with precise instructions despite succeeding at general tasks (Tripathni, et. al 2025). In this case, the walkthorugh LLM must balance specific pedagogical guidelines with strict JSON formatting. This happens across a particularly long prompt where instructions in the middle are more likely to be ignored (Liu, et al. 2023).

Another factor to consider with failure rates is the problem type and difficulty associated with the failure. Since this project used paired testing and their are 20 batches total, there should be 20 problems of each type in the dataset. Capitalizing on this, we can find underpreforming difficulties or problem types, utilizing their normalized values for comparison:



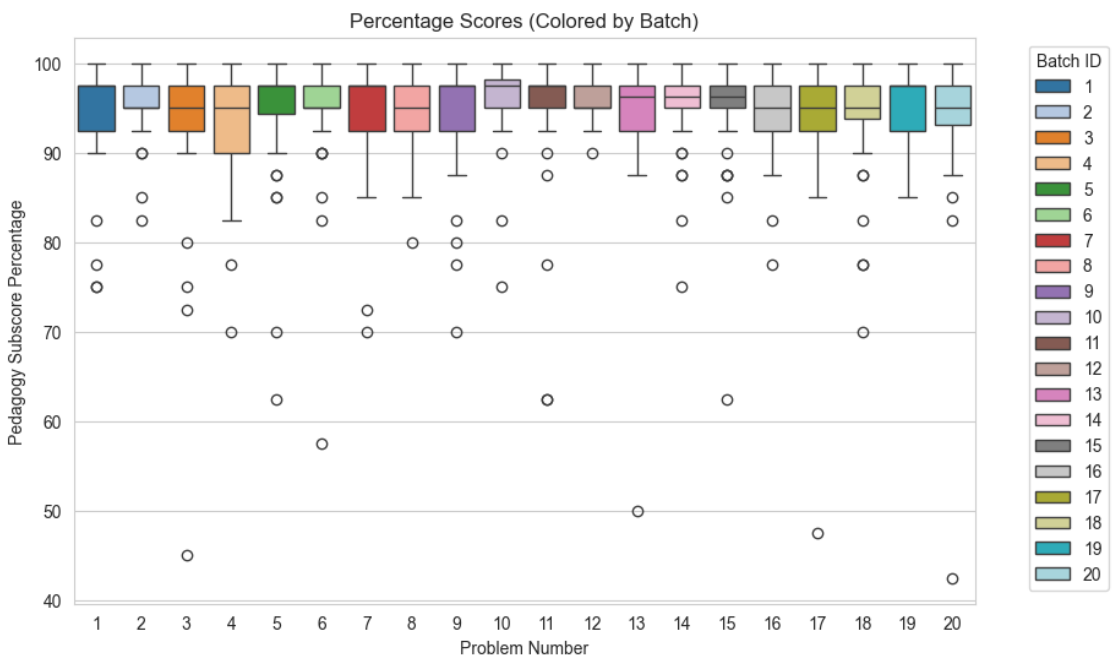
```
   problem_id problem_difficulty  \
5    bench_q6               hard
34  bench_q35               hard
18  bench_q19               hard
28  bench_q29               hard
33  bench_q34               hard

                                       problem_type  missing_counts
5                                          jacobian               3
34                           eigenvalues-eigenvectors               3
18                                 matrix-involutions               2
28                              line-integrals-vector               2
33  constrained optimization with inequality const…               2
```

Here we can see that the problems with highest failure are all of hard difficulty. These often involve complex, multi-step reasoning, which diverts attention away from formatting and leads to errors. This indicates that difficulty has a larger effect on failures than prompt alone.

Overcorrection Analysis The previous paper indicated that the prompt-revision LLM may overcorrect for minor mistakes in pedagogy: - Test the vailidity of this claim with the new paired testing scheme - Identify patterns in pedagogy score data

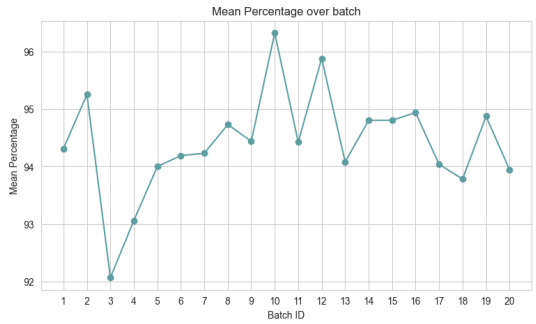
First we will look at how the scores change throughout each batch:

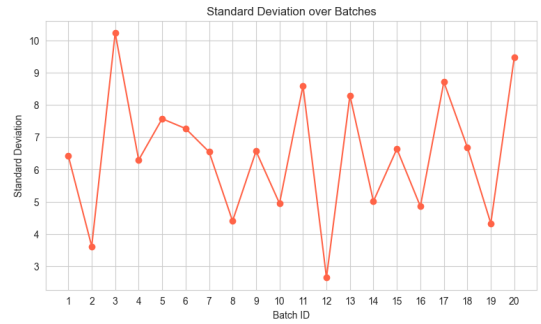
```
<matplotlib.legend.Legend at 0x122390ad0>
```



This boxplot shows that each prompt offers a unique median, IQR, and outliers. However, there

is no discernable pattern for the mean percentages or ranges batch over batch. To get a better perspective of the variance and flxuation in the mean, we will look at the direct graphs of mean and standard deviation over each batch:





The mean percentage graph shows that continuous improvement is rare. Instead, after large improvements in score, there is usually an equally strong decline (see batches 2 to 3, 10 to 11). This is likely due to the overcorrection recognised in the previous paper. Gradual improvements seem to have more consistent improvment over time, albeit with less overall increase than the spikes (see batches 4 through 8).

Additionally, these graphs do indicate that average score and standard deviation are inversely proportional. For instance, batch three has an all-time low score, while having the highest standard deviation. Meanwhile, batch 12 has the second highest score, while having the lowest standard deviation. To confirm this relationship, we'll take a look at the correlation between the score and the standard deviation.

```
np.float64(-0.7482340260997168)
```

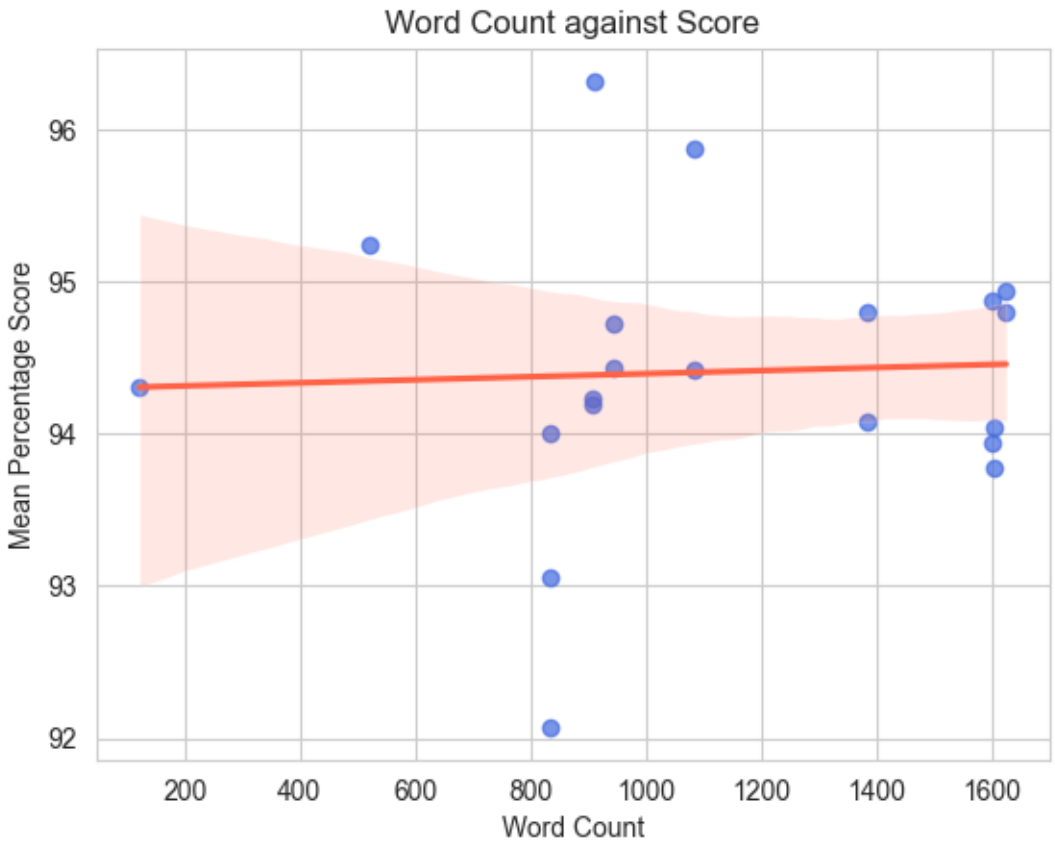
There is a strong inverse correlation between the score and the standard deviation (-0.748). Essentially, as the score of a response improves, it is more likely that these results will be consistent. This indicates that a quality prompt not only can improve the score of responses, but also the consistency of those responses.

Prompt Analysis:

- Does word count have a any affect on performance?

We will compare the word count to the mean percentage score, using a linear regression map and correlation to determine the effect.

```
Text(0.5, 1.0, 'Word Count against Score')
```




```
np.float64(0.04652051567957598)
```

Based on the correlation value (~0.047) and the correlation graph, the length of a prompt does not significantly affect the overall score of the response. There is not a statistically significant correlation between these two variables.

```
Text(0, 0.5, 'Missing Count')
```

Looking back to the error section, it is important to check if word count has a significant effect on message failure rates. Based on current data, the word count does not seem to have a significant impact on the failure rate of responses. To make sure there isn't a signficant connection, the correlation will be computed:

```
np.float64(0.06495428660845032)
```

A correlation of ~0.065 confirms that the correlation between word count and JSON errors is statistically insigificant.

1.0.6 Conclusion and Discussion:

Is there a pattern in overcorrection? Although there is overcorrection, there is no regular pattern in the mean or standard variance of response scores. There is, however, an notable trend where large increases are immediately followed by large decreases. Still, this behavior has no regular interval and seems reactionary, at least over 20 batches.

During the overcorrection analysis, we discovered that the score of responses are negatively correlated with the standard deviation of responses. This indicates that prompts that score well are also more consistent in their quality.

Does word count affect score and data loss? The word count does not have a major effect on the scoring of walkthrough responses or data loss thorugh JSON formatting errors. Rather, it was found that difficulty has a greater effect on data loss. This is because the LLM is attempting to do multiple seperate but specific tasks: Formatting answers in JSON, providing quality responses, and reasoning through complex, multistep problems. In cases where there was data loss, the LLM compromised JSON formatting while attempting to adhere to the rubric's pedagogical requirements and solving the problem correctly.

Choosing the Production Prompt: There are two main factors when choosing an adequate production prompt: quality responses, and consistency. For this reason, the prompt associated with batch 2 will be implemented as the new system prompt on MathTrain. It carries the second lowest standard deviation (most consistent results), while maintaining the third highest mean score. Combined with a 0% failure rate in these simulated tests, this prompt will ensure consistent, high-quality responses that render properly on the website.

Further Research: With a system in place to continuously revise respones for one model, we should now seek to track the response to feedback between different langauge models. This includes mixing different language models for different purposes in the generation pipeline (i.e. Gemini for Solutions, ChatGPT for grading, Claude for Revision, etc.). This could find a better balance between consistency in grading, quality of responses, and proper revision. Augmenting the pipeline to cycle through different combinations could

Additionally, we should seek to continue progress with the pipeline development process itself. Then natural next step is to create a context management protocol in which the grading and revision models are able to use past prompts and grades as context. Perhaps the grading model will provide better feedback that what is currently offered. It could also allow the editing model to not repeat mistakes from previous iterations.

1.0.7 References:

Tripathi, V., Allu, U., & Ahmed, B. (2025). The Instruction Gap: LLMs get lost in Following Instruction. arXiv (Cornell University). https://doi.org/10.48550/arxiv.2601.03269

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 12, 157–173. https://doi.org/10.1162/tacl_a_00638